



DAAS API 0.21.3

Reference Manual

January 5, 2026

1 Data Structure Documentation	1
1.1 DaasAPI Class Reference	1
1.1.1 Member Function Documentation	2
1.2 DaaSEvent Class Reference	56
1.3 DDO Class Reference	56
1.4 dperf_info_result Struct Reference	56
1.5 IDaasApiEvent Class Reference	56
1.6 IDepot Class Reference	56
1.7 nodestate_t Struct Reference	56
1.8 Vector< T > Class Template Reference	56

1 Data Structure Documentation

1.1 DaasAPI Class Reference

Public Member Functions

- const char * [getVersion](#) ()
- const char * [getBuildInfo](#) ()
- const char * [listAvailableDrivers](#) ()
- daas_error_t [doEnd](#) ()
- daas_error_t [doReset](#) ()
- daas_error_t [doInit](#) (din_t sid, din_t din)
- daas_error_t [doPerform](#) (performs_mode_t mode)
- daas_error_t [enableDriver](#) (link_t driver_id, const char *local_uri)
- nodestate_t [getStatus](#) ()
- void [setAcceptRequestsLevel](#) (int policy_level)
- bool [storeConfiguration](#) (IDepot *storage_interface)
- bool [loadConfiguration](#) (IDepot *storage_interface)
- bool [doStatisticsReset](#) ()
- uint64_t [getSystemStatistics](#) (syscode_t label)
- daas_error_t [map](#) (din_t din)
- daas_error_t [map](#) (din_t din, link_t link, const char *suri)
- daas_error_t [map](#) (din_t din, link_t link, const char *suri, const char *skey)
- daas_error_t [remove](#) (din_t din)
- daas_error_t [discovery](#) ()
- daas_error_t [discovery](#) (link_t link)
- void [setDiscoveryState](#) (discovery_state_t mode)
- dinlist_t [listNodes](#) ()
- daas_error_t [locate](#) (din_t din, int timeout=1000)
- daas_error_t [sendStatus](#) (din_t din)



- const `nodestate_t` & `status` (`din_t` din)
- const `nodestate_t` & `fetch` (`din_t` din, `uint16_t` opts)
- `uint64_t` `getSyncedTimestamp` ()
- const `nodestate_t` & `unlock` (`din_t` din, const char *skey)
- const `nodestate_t` & `lock` (const char *skey, unsigned policy_)
- const `nodestate_t` & `syncNode` (`din_t` din, unsigned timezone)
- const `nodestate_t` & `syncNet` (`din_t` din, unsigned bubble_time)
- void `setATSMMaxError` (`int32_t` error)
- bool `use` (`din_t` din)
- bool `end` (`din_t` din)
- unsigned `send` (`din_t` din, unsigned char *outbound, unsigned size)
- unsigned `received` (`din_t` din)
- unsigned `receive` (`din_t` din, unsigned char &inbound, unsigned max_size)
- `tsetlist_t` & `listTypesets` ()
- `daas_error_t` `pull` (`din_t` din, `DDO` **inboundDDO)
- `daas_error_t` `push` (`din_t` din, `DDO` *outboundDDO)
- `daas_error_t` `availablesPull` (`din_t` din, `uint32_t` &count)
- `daas_error_t` `addTypeset` (const `uint16_t` typeset_code, const typeset_fun fun)
- `daas_error_t` `frisbee` (`din_t` din)
- `daas_error_t` `frisbeeCMP` (`din_t` din, `uint32_t` timeout, `uint32_t` retry)
- `daas_error_t` `frisbeeDPERF` (`din_t` din, `uint32_t` sender_pkt_total=10, `uint32_t` block_size=1024 *1024, `uint32_t` sender_trip_period=0)
- `dperf_info_result` `getFrisbeeResultDPERF` ()
- `daas_error_t` `setDDOPolicy` (`ddo_policy_t` policy)
- `daas_error_t` `unbindNetwork` ()

1.1.1 Member Function Documentation



addTypeset()

```
daas_error_t addTypeset (  
    const uint16_t typeset_code,  
    const typeset_fun fun)
```

Maps a user-defined typeset to a handler function. If a [DDO](#) with the specified typeset is received, the provided handler function will be called.

Parameters

typeset_code	the code of the typeset to add
fun	The function to handle received DDOs of this typeset

Note

The registered typeset does not trigger the received [DDO](#) event.

Returns

ERROR_NONE on success, or an error code on failure.



Example:

```
#define hello_worlds_typeset 120
#define random_calculation_typeset 122

node.doPerform(PERFORM_CORE_THREAD);

node.addTypeset(hello_worlds_typeset, [](din_t din){
    DDO *pk;
    if (node.pull(din, &pk) == ERROR_NONE){
        unsigned char * payload = pk->getPayloadPtr();
        printf("Hello World from DIN %llu: %s\n", din, payload);
        delete pk;
    }
});

node.addTypeset(random_calculation_typeset, [](din_t din){
    DDO *pk;
    if (node.pull(din, &pk) == ERROR_NONE){
        unsigned char * payload = pk->getPayloadPtr();
        int limit = *((int*)payload);
        int random_number = rand() % limit;
        printf("Random number from DIN %llu with limit %d: %d\n", din, limit, random_number);
        delete pk;
    }
});

/*
    Sender side:

    DDO hello_ddo;
    hello_ddo.setTypeset(hello_worlds_typeset);
    node.push(remoteDIN, &hello_ddo);

    On the receiving side, the handler for hello_worlds_typeset will be triggered and print
    the message.
*/
```



availablesPull()

```
daas_error_t availablesPull (  
    din_t din,  
    uint32_t & count)
```

Checks if there are available DDOs from a remote node.

Parameters

din	DIN of the remote node to check for available DDOs
count	reference to a variable that will hold the number of available DDOs

Returns

ERROR_NONE on success, or an error code on failure.

See also
[pull](#)



discovery() [1/2]

```
daas_error_t discovery ()
```

Starts a discovery process to locate nodes in the network.
This process will send the enabled drivers URI to a sort of broadcast in order to auto-map this node to remote ones.

Note

The reception of this packet cannot be controlled, it can only be disabled.

Parameters

none : starts discovery on all available links

Returns

ERROR_NONE on success, or an error code on failure.

See also

[doInit](#)



Example:

```
// In case of missing initialization, the node can generate din and sid automatically if
// discovery is used.
// The sid will be assigned by the node master of the network we are connecting to.
// The din will be generated automatically by the node itself and will be combined with some
// time related value given by the
// network we are connecting to.

if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
{
    fprintf(stdout, "Driver enabled successfully.\n");
}
else
{
    fprintf(stderr, "Failed to enable driver.\n");
    return -1;
}

node.doPerform(PERFORM_CORE_THREAD);

// Send discovery beacons for 10 times with 100ms interval
for (int i = 0; i < 10; i++){
    node.discovery();
    sleep(100); // Sleep 100ms between each beacon sent
}

// other logic down here..

// The discovery in this case will be only trasmitted though inet4 which is the only driver
// enabled.
```



discovery() [2/2]

```
daas_error_t discovery (  
    link_t link)
```

Starts a discovery process to locate nodes in the network.
This process will send the enabled drivers URI to a sort of broadcast in order to auto-map this node to remote ones.

Note

The reception of this packet cannot be controlled, it can only be disabled.

Parameters

link The communication technology to use for discovery. If not specified, all available links will be used.

Returns

ERROR_NONE on success, or an error code on failure.

See also

[doInit](#)



Example:

```
// In case of missing initialization, the node can generate din and sid automatically if
// discovery is used.
// The sid will be assigned by the node master of the network we are connecting to.
// The din will be generated automatically by the node itself and will be combined with some
// time related value given by the
// network we are connecting to.

if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
{
    fprintf(stdout, "Driver enabled successfully.\n");
}
else
{
    fprintf(stderr, "Failed to enable driver.\n");
    return -1;
}

node.doPerform(PERFORM_CORE_THREAD);

// Send discovery beacons for 10 times with 100ms interval
for (int i = 0; i < 10; i++){
    node.discovery();
    sleep(100); // Sleep 100ms between each beacon sent
}

// other logic down here..

// The discovery in this case will be only trasmitted though inet4 which is the only driver
// enabled.
```



doEnd()

```
daas_error_t doEnd ()
```

Releases resources and deactivates the node.

Parameters

none

Returns

ERROR_NONE on success, or an error code on failure.



doInit()

```
daas_error_t doInit (  
    din_t sid,  
    din_t din)
```

Initializes services and resources for the local node.

Parameters

sid SID of the local node

din DIN of the local node

Returns

ERROR_NONE on success, or an error code on failure.



Example:

```
#include <stdio.h>
#include <daas.hpp>

bool exit = false;
DaasAPI node = DaasAPI();

int main() {
    din_t localSID = 100;
    din_t localDIN = 200;

    din_t remoteDIN = 201;

    /* Note that the initialization of the node is not strictly necessary anymore.
     * In case of missing din and/or sid, the node can generate those informations.
     * The sid will be assigned by the node master of the network we are connecting to.
     * The din will be generated automatically by the node itself and will be combined with some
     * time related value given by the network we are connecting to.
     *
     * However, if the initialization is not performed, the node will not be able to communicate
     * until it connects to a network through the discovery feature.
     */

    if (node.doInit(localSID, localDIN) == ERROR_NONE)
    {
        fprintf(stdout, "Node initialized with SID: %llu, DIN: %llu\n", localSID, localDIN);
    }
    else
    {
        fprintf(stderr, "Failed to initialize node with SID: %llu, DIN: %llu\n", localSID,
            localDIN);
        return -1;
    }

    if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
    {
        fprintf(stdout, "Driver enabled successfully.\n");
    }
    else
    {
        fprintf(stderr, "Failed to enable driver.\n");
        return -1;
    }

    if (node.map(remoteDIN, _LINK_INET4, "127.0.0.1:2021")) {
        fprintf(stdout, "Remote DIN %u mapped successfully.\n", remoteDIN);
    } else {
        fprintf(stderr, "Failed to map remote DIN %u.\n", remoteDIN);
        return -1;
    }

    node.doPerform(PERFORM_CORE_THREAD);

    while (!exit) {
        // Perform the operation here
    }

    return 0;
}
```



doPerform()

```
daas_error_t doPerform (  
    performs_mode_t mode)
```

Executes the node's main processing loop in either real-time or multi-threading mode. It manages the node's internal state, including network communication, data processing, and synchronization with other nodes. In **multi-threading** mode, it spawns internal threads to handle processing.

Parameters

mode

- **PERFORM_CORE_THREAD** for multi-threading mode
- **PERFORM_CORE_NO_THREAD** for real-time mode

Returns

ERROR_NONE on success, or an error code on failure.

Note

This function **must** be called if the node has to be operational. In **multi-threading** mode it has to be called only once, while in **real-time** mode it has to be called cyclically.

See also

[doInit](#)



doReset()

```
daas_error_t doReset ()
```

Resets the local node and clears all resources. It disconnects also from the network that it currently is.

Parameters

none

Returns

ERROR_NONE on success, or an error code on failure.



doStatisticsReset()

```
bool doStatisticsReset ()
```

Resets the system's statistics data.

Parameters

none

Returns

true if the reset was successful, false otherwise.



enableDriver()

```
daas_error_t enableDriver (
    link_t driver_id,
    const char * local_uri)
```

Configure driver for network technology (links)

Parameters

- driver_id** the communication technology to enable:
- **_LINK_INET4**
 - **_LINK_BLUETOOTH**
 - **and** others as defined in `link_t`
- local_uri** the physical address of the local node:
- For **_LINK_INET4**: a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000")
 - For **_LINK_BLUETOOTH**: a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")

Returns

ERROR_NONE on success, or an error code on failure.

See also
[doInit](#)



end()

```
bool end (  
    din_t din)
```

Ends a real-time session with a remote node.

Parameters

din DIN of the remote node to end the session with

Warning

Not implemented yet!

Returns

true if the RT session was successfully ended, false otherwise.



fetch()

```
const nodestate_t & fetch (  
    din_t din,  
    uint16_t opts)
```

Fetches the status of a remote node and updates its status in the local instance.

Parameters

din DIN of the remote node to fetch the status from

opts options for fetching

Returns

the `nodestate_t` of the remote node after fetching.



frisbee()

```
daas_error_t frisbee (  
    din_t din)
```

Pings a remote node to check its availability.

Parameters

din DIN of the remote node to ping

Returns

ERROR_NONE on success, or an error code on failure.



frisbeeDPERF()

```
daas_error_t frisbeeDPERF (
    din_t din,
    uint32_t sender_pkt_total = 10,
    uint32_t block_size = 1024 * 1024,
    uint32_t sender_trip_period = 0)
```

Measures the performance of data transfer to a remote node.

Parameters

din	DIN of the remote node to ping
sender_pkt↔ _total	total number of packets to send
block_size	size of each packet in bytes
sender_trip↔ _period	time period between each packet sent (in milliseconds)

Returns

ERROR_NONE on success, or an error code on failure.



frisbeeICMP()

```
daas_error_t frisbeeICMP (  
    din_t din,  
    uint32_t timeout,  
    uint32_t retry)
```

Pings a remote node with a specified timeout and retry count.

Parameters

din	DIN of the remote node to ping
timeout	maximum time to wait for a reply (in milliseconds)
retry	number of retries if no reply is received

Returns

ERROR_NONE on success, or an error code on failure.



getBuildInfo()

```
const char * getBuildInfo ()
```

Retrieves build information of the DaaS API library.

Parameters

none

Returns

A string containing build information such as compiler version and build configuration.



getFrisbeeResultDPERF()

```
dperf_info_result getFrisbeeResultDPERF ()
```

Returns the result of a frisbee performance test.

Parameters

none

Returns

a **dperf_info_result** structure containing the performance test results:

- **sender_first_timestamp**: timestamp of the first packet sent by the sender
- **local_end_timestamp**: timestamp of the last packet received by the sender
- **remote_first_timestamp**: timestamp of the first packet received by the receiver
- **remote_last_timestamp**: timestamp of the last packet received by the receiver
- **remote_pkt_counter**: number of packets received by the receiver
- **remote_data_counter**: total data received by the receiver in bytes



getStatus()

```
nodestate_t getStatus ()
```

Returns the status of the local node.

Parameters

none

Returns

the status of the local node.

- This includes hardware version, linked channels, synchronization status, security policy, and more.



getSyncedTimestamp()

```
uint64_t getSyncedTimestamp ()
```

Time with ATS correction used to be able to communicate inside the DaaS network.

Parameters

none

Attention

This function returns the synchronized timestamp of the local node, only when connected to a DaaS network and ATS is in sync. If the node is not synchronized, the function returns the local system time without ATS correction.

Returns

the synced timestamp of the local node.



getSystemStatistics()

```
uint64_t getSystemStatistics (  
    syscode_t label)
```

Returns the statistics of the local node for a specific system code.

Parameters

label the system code to get statistics for (e.g., `_cor_dme_sended`)

Returns

the system statistics for the given label.



getVersion()

```
const char * getVersion ()
```

Retrieves the version of the DaaS API library.

Parameters

none

Returns

A string representing the version of the DaaS API library.



listAvailableDrivers()

```
const char * listAvailableDrivers ()
```

Lists all available drivers in the DaaS stack for this node.

Parameters

none

Returns

A semicolon-separated string containing the list of available drivers.



listNodes()

```
dinlist_t listNodes ()
```

Returns map entries (known nodes) in the local instance.

Parameters

none

Returns

a list of known nodes (din_t) in the local instance.



listTypesets()

```
tsetlist_t & listTypesets ()
```

Returns a list of user-defined typesets.

Parameters

none

Returns

a reference to the list of user-defined typesets.

See also
[addTypeset](#)

Note
The list is of type `tsetlist_t`, which is a [Vector](#) of `typeset`



loadConfiguration()

```
bool loadConfiguration (  
    IDepot * storage_interface)
```

Loads the configuration from the specified storage interface.

Parameters

**storage_↔
interface** the interface to load the configuration from (e.g., a file system, database, etc.)

Returns

true if the configuration was loaded successfully, false otherwise.



Example:

```
// Simple implementation of IDepot for demonstration purposes
class device_storage : public IDepot
{
    // Open the partition handler
    bool open(bool read_mode)
    {
        if (f == nullptr)
        {
            const char *mode = read_mode ? "rb" : "wb";
            f = fopen("storage.txt", mode);
        }

        return f != nullptr;
    }

    // Close the partition handler
    bool close()
    {
        if (f && fclose(f) == 0)
            f = nullptr;

        return f == nullptr;
    }

    // Deletes all the record saved
    bool clearSpace() {}

    // Get how much space is left on the device
    bool getSpaceInfo() {}

    // Trash a specific entry
    bool trash(unsigned oKey) {}

    // Save map to buffer
    unsigned save(unsigned oKey, unsigned char *pdata, unsigned dsize)
    {
        if (f)
            return fwrite(pdata, dsize, 1, f);

        return 0;
    }

    // Load map data from buffer
    unsigned load(unsigned oKey, unsigned char *pdata, unsigned dsize)
    {
        if (f)
            return fread(pdata, dsize, 1, f);

        return 0;
    }

private:
    FILE *f = nullptr;
};

bool exit = false;
device_storage storage;
DaasAPI node = DaasAPI();

int main() {

    // load configuration from storage
    node.loadConfiguration(&storage);

    node.doPerform(PERFORM_CORE_THREAD);

    // Perfom operation here

    // To store configuration
    node.storeConfiguration(&storage);

    return 0;
}
```



locate()

```
daas_error_t locate (  
    din_t din,  
    int timeout = 1000)
```

It starts a process to locate the node if it is not inside the known table.

Parameters

din_ DIN of the node to locate

timeout Maximum time to wait for the node to be located (in milliseconds). Default is 1000 ms.

Returns

ERROR_NONE if the node is known, or an error code if it is not.

See also
[pull](#)



lock()

```
const nodestate_t & lock (  
    const char * skey,  
    unsigned policy_)
```

Set SKEY and security policy for local node

Parameters

skey the security key to set

policy_ the security policy to set

Warning

Not implemented yet!

Returns

the `nodestate_t` of the local node after setting the security key and policy.



map() [1/3]

```
daas_error_t map (  
    din_t din)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din DIN of the node to map

Returns

ERROR_NONE on success, or an error code on failure.

See also
[dolnit](#)



map() [2/3]

```
daas_error_t map (
    din_t din,
    link_t link,
    const char * suri)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din	DIN of the node to map
link	the communication technology to use: • _LINK_INET4 • _LINK_BLUETOOTH • and others as defined in <code>link_t</code>
suri	the physical address of the node: • For _LINK_INET4: a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000") • For _LINK_BLUETOOTH: a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")

Returns

ERROR_NONE on success, or an error code on failure.

Note

The communication links supported depend on the build configuration and enabled drivers. Those can be listed using the `listAvailableDrivers()` function.

See also
[doInit](#)



map() [3/3]

```
daas_error_t map (
    din_t din,
    link_t link,
    const char * suri,
    const char * skey)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din	DIN of the node to map
link	the communication technology to use: • _LINK_INET4 • _LINK_BLUETOOTH • and others as defined in <code>link_t</code>
suri	the physical address of the node: • For _LINK_INET4: a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000") • For _LINK_BLUETOOTH: a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")
skey	the security key for the node. Currently not implemented.

Returns

ERROR_NONE on success, or an error code on failure.

Note

The communication links supported depend on the build configuration and enabled drivers. Those can be listed using the `listAvailableDrivers()` function.

See also
[doInit](#)



pull()

```
daas_error_t pull (
    din_t din,
    DDO ** inboundDDO)
```

Pulls a DDO from a remote node.

Parameters

din DIN of the remote node to pull data from

inboundDDO pointer to a DDO pointer that will hold the pulled DDO

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
node.doPerform(PERFORM_CORE_THREAD);

DDO* ddo;
while (!exit) {
    uint32_t n_pulls = 0;
    if (node.locate(remoteDIN) == ERROR_NONE && node.availablesPull(remoteDIN, n_pulls) ==
        ERROR_NONE && n_pulls > 0)
    {
        if (node.pull(remoteDIN, &ddo) == ERROR_NONE)
        {
            fprintf(stdout, "Pulled DDO with typeset: %lu, origin: %llu, timestamp: %llu\n",
                ddo->getTypeset(), ddo->getOrigin(), ddo->getTimestamp());
        }
        else
        {
            fprintf(stderr, "Failed to pull DDO.\n");
        }
    }
}
```



push()

```
daas_error_t push (
    din_t din,
    DDO * outboundDDO)
```

Pushes a DDO to a remote node.

Parameters

din DIN of the remote node to send data to

outboundDDO ↔ pointer to the DDO to send

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
node.doPerform(PERFORM_CORE_THREAD);

DDO ddo;

ddo.setPayload("Hello world!\n", 14);
ddo.setTypeSet(0);

while (!exit) {
    if (node.locate(remoteDIN) == ERROR_NONE && node.push(remoteDIN, &ddo) == ERROR_NONE)
    {
        fprintf(stdout, "DDO sent correctly");
    }else{
        fprintf(stderr, "Error during push of the DDO");
    }
}
```



receive()

```
unsigned receive (  
    din_t din,  
    unsigned char & inbound,  
    unsigned max_size)
```

Receives data from a remote node in a real-time session.

Parameters

din	DIN of the remote node to receive data from
inbound	reference to a variable that will hold the received data
max_size	maximum size of data to receive

Warning

Not implemented yet!

Returns

the size of data received.



received()

```
unsigned received (  
    din_t din)
```

Checks if there is data available from a remote node in a real-time session.

Parameters

din DIN of the remote node to check for data

Warning

Not implemented yet!

Returns

the size of data received.



remove()

```
daas_error_t remove (  
    din_t din)
```

Removes a node from the local instance and cleans up associated resources.

Parameters

din DIN of the node to remove

Returns

ERROR_NONE on success, or an error code on failure.



send()

```
unsigned send (  
    din_t din,  
    unsigned char * outbound,  
    unsigned size)
```

Sends data to a remote node in a real-time session.

Parameters

din	DIN of the remote node to send data to
outbound	pointer to the data to send
size	size of the data to send

Warning

Not implemented yet!

Returns

the size of data sent.



sendStatus()

```
daas_error_t sendStatus (  
    din_t din)
```

Send the local node's status to a remote node.

Parameters

din DIN of the remote node to send the status to

Returns

ERROR_NONE on success, or an error code on failure.



setAcceptRequestsLevel()

```
void setAcceptRequestsLevel (  
    int policy_level)
```

The function sets whether the local node should accept all incoming requests, including those from unknown DINs, known din from different network or only from din inside the same network.

Parameters

policy_level

- **0**: accept requests only from known DINs inside the same network.
- **1**: accept requests from known DINs from different networks and inside the same network.
- **2**: accept requests from all DINs, including unknown ones

Note

Default policy_level is **1**. During the discovery, the policy_level is temporary set to **2** since the node has to accept all the incoming request. Right after the discovery process is compleated or goes out of time, the policy_level is set back to the previous value.



setATSMaxError()

```
void setATSMaxError (  
    int32_t error)
```

Set the maximum error allowed for ATS synchronization. This value defines the acceptable time deviation between the local node and remote nodes in the network. The ATS mechanism uses this parameter to ensure that all nodes maintain a consistent time reference within the specified error margin.

Note

By default, this value is set to **100** milliseconds.

Parameters

error	the maximum error in milliseconds
--------------	-----------------------------------



setDDOPolicy()

```
daas_error_t setDDOPolicy (
    ddo_policy_t policy)
```

Sets the DDO policy for the local node. This policy determines how to manage the unsuccessful sending of DDOs.

Parameters

policy

the DDO policy to set (ddo_policy_t)

- **ddo_policy_skip_on_failure:** Skip sending the DDO on failure and deletes it from the sending buffer.
- **ddo_policy_retry_on_failure:** Retry sending the DDO at every doPerform cycle.
- **ddo_policy_exponential_backoff_retry_on_failure:** Retry sending the DDO with exponential backoff on failure. Currently, the backoff starts at 1 minute and doubles with each failure, up to a maximum of 1 hour.

Note

The default policy is **ddo_policy_skip_on_failure**. If the node with mode **ddo_policy_exponential_backoff_retry_on_failure** receives a DDO from the remote node, the retry timer for that DDO is reset and sending will be retried immediately in the next doPerform cycle.

Attention

The retry policies may change the order of DDOs being sent, especially if multiple DDOs are queued for the same remote node. For example, if DDO1 fails to send and is set to retry later, while DDO2 is successfully sent in the next cycle, DDO2 may be sent before DDO1.

Returns

ERROR_NONE on success, or an error code on failure.



setDiscoveryState()

```
void setDiscoveryState (  
    discovery_state_t mode)
```

Configures how the node interacts with the network discovery process. Depending on the selected mode, the node can search for existing networks, host new ones, or remain invisible to other nodes.

Parameters

mode

The discovery activation mode:

- **discovery_off**: Completely disables discovery. The node will neither send nor respond to discovery beacons.
- **discovery_sender_only**: The node actively searches for existing networks but will not host or respond to discovery packets from others.
- **discovery_receiver_only**: The node acts as a host, assigning network parameters to others, but will not seek to join external networks.
- **discovery_full**: Enables full bidirectional discovery (both seeking and hosting/joining networks).

Note

By default, the node is set to **discovery_sender_only** mode upon initialization.



status()

```
const nodestate_t & status (  
    din_t din)
```

Fetches the status of a remote node.

Parameters

din DIN of the remote node to fetch the status from

Returns

the `nodestate_t` of the remote node.



storeConfiguration()

```
bool storeConfiguration (
    IDepot * storage_interface)
```

Saves the current configuration to the specified storage interface.

Parameters

**storage_↔
interface** the interface to save the configuration (e.g., a file system, database, etc.)

Returns

true if the backup was successful, false otherwise.

See also

[loadConfiguration](#)



syncNet()

```
const nodestate_t & syncNet (  
    din_t din,  
    unsigned bubble_time)
```

Set the local system time on remote node *din* and synchronize ATS

Parameters

din	DIN of the remote node to synchronize
bubble_time	max error allowed for synchronization in milliseconds

Warning

Not implemented yet!

Returns

the `nodestate_t` of the remote node after synchronization.



syncNode()

```
const nodestate_t & syncNode (  
    din_t din,  
    unsigned timezone)
```

Set the local system time on remote node *din* and synchronize ATS.

Parameters

din DIN of the remote node to synchronize

timezone the timezone offset in seconds

Warning

Not implemented yet!

Returns

the `nodestate_t` of the remote node after synchronization.



unbindNetwork()

```
daas_error_t unbindNetwork ()
```

Unbinds the local node from the current network, closing all the open channels. The mapping table is preserved, so the node can rebind to the same network later without needing to remap known nodes.

Warning

After unbinding, the node will not be able to communicate with some nodes of the previous network (it depends on their [setAcceptRequestsLevel](#)).

Parameters

none

Returns

ERROR_NONE on success, or an error code on failure.



unlock()

```
const nodestate_t & unlock (  
    din_t din,  
    const char * skey)
```

Unlocks a remote node by setting its security key.

Parameters

din DIN of the remote node to unlock

skey the security key to set

Warning

Not implemented yet!

Returns

the `nodestate_t` of the remote node after unlocking.



use()

```
bool use (  
    din_t din)
```

Starts a real-time session with a remote node.

Parameters

din DIN of the remote node to start the session with

Warning

Not implemented yet!

Returns

true if the RT session was successfully started, false otherwise. (OPEN CONNECTION!!!!)



- 1.2 DaaSEvent Class Reference**
- 1.3 DDO Class Reference**
- 1.4 dperf_info_result Struct Reference**
- 1.5 IDaasApiEvent Class Reference**
- 1.6 IDepot Class Reference**
- 1.7 nodestate_t Struct Reference**
- 1.8 Vector< T > Class Template Reference**

